

```
C:\ayush\crypto\lab1.exe
Enter text: iamayush
Enter key (0-25): 3
Cipher: LDPDBXVK
Plain : IAMAYUSH
```

```
C:\ayush\crypto\lab2.exe
1. Encrypt
2. Decrypt
Choose: 1
Enter key: KEY
Enter text: PLAIN
Ciphertext: QIKNSA
```

```
C:\ayush\crypto\lab2.exe
1. Encrypt
2. Decrypt
Choose: 2
Enter key: KEY
Enter text: QIKNSA
Plaintext (raw): PLAINX
```

```
C:\ayush\crypto\lab3.exe
Enter 2x2 key matrix (a b c d): 3 3 2 5
1) Encrypt
2) Decrypt
Choose: 1
Enter text: HELLO
Ciphertext: HIOZHN
```

```
C:\ayush\crypto\lab3.exe
Enter 2x2 key matrix (a b c d): 3 3 2 5
1) Encrypt
2) Decrypt
Choose: 2
Enter text: HIOZHN
Plaintext: HELLOX
```

```
C:\ayush\crypto\lab4.exe
Vigenere Cipher
1) Encrypt
2) Decrypt
Choose: 1
Enter text: ATTACK AT DAWN
Enter key: LEMON
Ciphertext: LXFOPVEFRNHR
```

```
C:\ayush\crypto\lab4.exe
Vigenere Cipher
1) Encrypt
2) Decrypt
Choose: 2
Enter text: LXFOPVEFRNHR
Enter key: LEMON
Plaintext: ATTACKATDAWN
```

```
C:\ayush\crypto\lab5.exe
One-Time Pad (OTP)
1) Encrypt
2) Decrypt
Choose: 1
Enter text: HELLO
Enter key (same length as text): XMCKL
Ciphertext: EQNVZ
```

```
C:\ayush\crypto\lab5.exe
One-Time Pad (OTP)
1) Encrypt
2) Decrypt
Choose: 2
Enter text: EQNVZ
Enter key (same length as text): XMCKL
Plaintext: HELLO
```

```
C:\ayush\crypto\lab6.exe
Vernam Cipher (Binary XOR)
1) Encrypt
2) Decrypt
Choose: 1
Enter plaintext: HELLO
Enter key (same length): XMCKL

Step-by-step XOR (8-bit):
P_char P_bin      K_char K_bin      C_bin
H      01001000   X      01011000   00010000
E      01000101   M      01001101   00001000
L      01001100   C      01000011   00001111
L      01001100   K      01001011   00000111
O      01001111   L      01001100   00000011

Ciphertext (numbers): 16 8 15 7 3
(Use these numbers for Decrypt option)
```

```
C:\ayush\crypto\lab6.exe
Vernam Cipher (Binary XOR)
1) Encrypt
2) Decrypt
Choose: 2
Enter ciphertext numbers (space separated): 16 8 15 7 3
Enter key (same length as number count): XMCKL

Step-by-step XOR (8-bit):
C_num C_bin      K_char K_bin      P_bin
16     00010000   X      01011000   01001000
8      00001000   M      01001101   01000101
15     00001111   C      01000011   01001100
7      00000111   K      01001011   01001100
3      00000011   L      01001100   01001111

Plaintext: HELLO
```

Lab8: Source code

```
#include <bits/stdc++.h>

using namespace std;

int main()
{
    int S1[4][16] = {
        {14, 4, 13, 1, 2, 15, 11, 8, 3, 10, 6, 12, 5, 9, 0, 7},
        {0, 15, 7, 4, 14, 2, 13, 1, 10, 6, 12, 11, 9, 5, 3, 8},
        {4, 1, 14, 8, 13, 6, 2, 11, 15, 12, 9, 7, 3, 10, 5, 0},
        {15, 12, 8, 2, 4, 9, 1, 7, 5, 11, 3, 14, 10, 0, 6, 13}
    };

    string bits;

    cout << "Enter 6-bit input : ";

    cin >> bits;

    if (bits.length() != 6)
        return cout << "Invalid input: need exactly 6 bits\n", 0;

    for (char c : bits)
        if (c != '0' && c != '1')
            return cout << "Invalid input: only characters '0' or '1' allowed", 0;

    int row = (bits[0] - '0') * 2 + (bits[5] - '0');
    int col = (bits[1] - '0') * 8 + (bits[2] - '0') * 4 + (bits[3] - '0') * 2 + (bits[4] - '0');
    int value = S1[row][col];

    string output = "";

    for (int i = 3; i >= 0; i--)
        output += char(((value >> i) & 1) + '0');

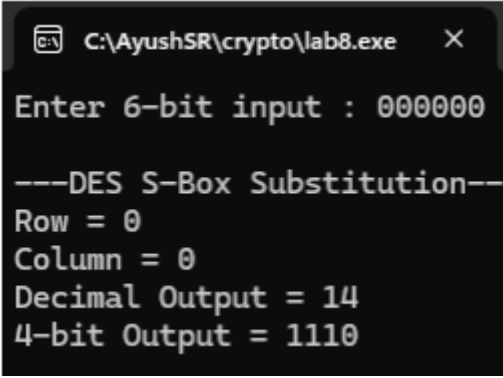
    cout << endl

    << "---DES S-Box Substitution---" << endl

    << "Row = " << row << endl
```

```
<< "Column = " << col << endl  
<< "Decimal Output = " << value << endl  
<< "4-bit Output = " << output << endl;  
return 0;  
}
```

Output:



```
C:\AyushSR\crypto\lab8.exe X  
Enter 6-bit input : 000000  
---DES S-Box Substitution---  
Row = 0  
Column = 0  
Decimal Output = 14  
4-bit Output = 1110
```

Lab9: Source code

```
#include <bits/stdc++.h>

using namespace std;

int
PC1[56]={57,49,41,33,25,17,9,1,58,50,42,34,26,18,10,2,59,51,43,35,27,19,11,3,60,52
,44,36,63,55,47,39,31,23,15,7,62,54,46,38,30,22,14,6,61,53,45,37,29,21,13,5,28,20,1
2,4};

int
PC2[48]={14,17,11,24,1,5,3,28,15,6,21,10,23,19,12,4,26,8,16,7,27,20,13,2,41,52,31,3
7,47,55,30,40,51,45,33,48,44,49,39,56,34,53,46,42,50,36,29,32};

int shifts[16]={1,1,2,2,2,2,2,2,1,2,2,2,2,2,2,1};

string permute(string input,int table[],int n){
    string output="";
    for(int i=0;i<n;i++){
        output+=input[table[i]-1];
    }
    return output;
}

string leftShift(string s,int count){
    while(count-->0)s=s.substr(1)+s[0];
    return s;
}

int main(){
    string key64;
    int rounds;

    cout<<"Enter 64-bit key : ";
    cin>>key64;

    cout<<"Enter number of rounds : ";
    cin>>rounds;

    if(key64.length()!=64 || rounds<1 || rounds>16)
```

```

return cout<<"Invalid input: key must be 64 bits & rounds 1-16",0;

for(char c:key64)if(c!='0'&&c!='1')

return cout<<"Invalid input: key must be binary string of 0/1 characters",0;

string key56=permute(key64,PC1,56);

string C=key56.substr(0,28);

string D=key56.substr(28,28);

cout<<endl<<"---DES Key Generation---"<<endl;

cout<<"Initial C : "<<C<<endl;

cout<<"Initial D : "<<D<<endl;

for(int i=0;i<rounds;i++){

C=leftShift(C,shifts[i]);

D=leftShift(D,shifts[i]);

string subkey=permute(C+D,PC2,48);

cout<<"Round "<<i+1<<" Subkey : "<<subkey<<endl;

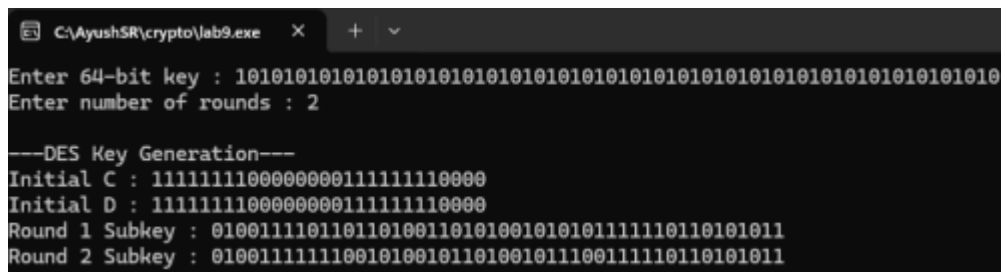
}

return 0;

}

```

Output:



```

C:\AyushSR\crypto\lab9.exe
Enter 64-bit key : 1010101010101010101010101010101010101010101010101010101010101010
Enter number of rounds : 2

---DES Key Generation---
Initial C : 1111111100000000111111110000
Initial D : 1111111100000000111111110000
Round 1 Subkey : 0100111101101101001101010010101111110110101011
Round 2 Subkey : 01001111110010100101101001011100111110110101011

```

Lab10: Source code

```
#include <bits/stdc++.h>

using namespace std;

long long normalize(long long x,long long n){

x%=n;

if(x<0)x+=n;

return x;

}

int main(){

long long a,b,n;

cout<<"Enter a, b and n : ";

cin>>a>>b>>n;

if(n<=0)

return cout<<"n must be positive",0;

long long A=normalize(a,n);

long long B=normalize(b,n);

cout<<endl<<"---Modular Arithmetic & Additive Inverses in Z"<<n<<"---"<<endl;

cout<<"("<<a<<" + "<<b<<"> mod "<<n<<" = "<<normalize(A+B,n)<<endl;

cout<<"("<<a<<" - "<<b<<"> mod "<<n<<" = "<<normalize(A-B,n)<<endl;

cout<<"("<<a<<" * "<<b<<"> mod "<<n<<" = "<<normalize(A*B,n)<<endl;

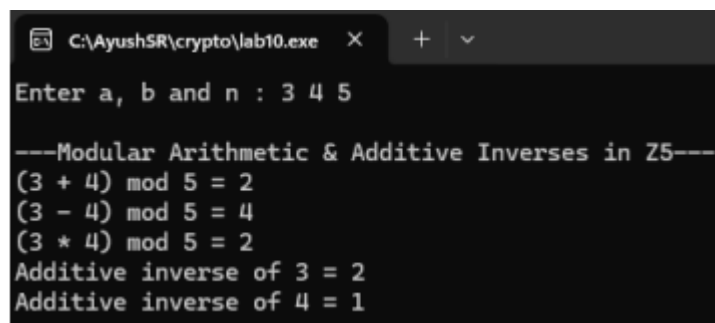
cout<<"Additive inverse of "<<a<<" = "<<normalize(-A,n)<<endl;

cout<<"Additive inverse of "<<b<<" = "<<normalize(-B,n)<<endl;

return 0;

}
```

Output:



```
C:\AyushSR\crypto\lab10.exe  X  +  v
Enter a, b and n : 3 4 5

---Modular Arithmetic & Additive Inverses in Z5---
(3 + 4) mod 5 = 2
(3 - 4) mod 5 = 4
(3 * 4) mod 5 = 2
Additive inverse of 3 = 2
Additive inverse of 4 = 1
```

Lab11: Source code

```
#include <bits/stdc++.h>

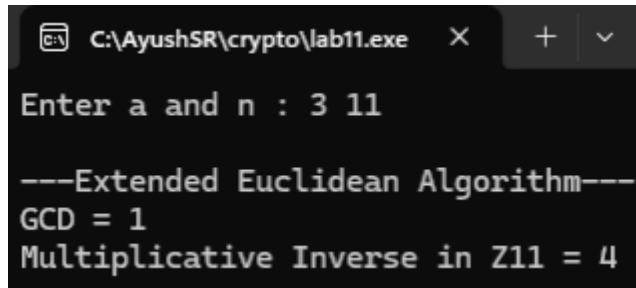
using namespace std;

long long extendedGCD(long long a,long long b,long long &x,long long &y){
    if(b==0){
        x=1;
        y=0;
        return a;
    }
    long long x1,y1;
    long long gcd=extendedGCD(b,a%b,x1,y1);
    x=y1;
    y=x1-(a/b)*y1;
    return gcd;
}

int main(){
    long long a,n,x,y;
    cout<<"Enter a and n : ";
    cin>>a>>n;
    if(n<=0)return cout<<"Invalid modulus: n must be positive",0;
    long long gcd=extendedGCD(a,n,x,y);
    cout<<endl<<"---Extended Euclidean Algorithm---"<<endl;
    cout<<"GCD = "<<gcd<<endl;
    if(gcd==1){
        long long inverse=(x%n+n)%n;
        cout<<"Multiplicative Inverse in Z"<<n<<" = "<<inverse<<endl;
    }
    else cout<<"Inverse does not exist: a & n are not coprime"<<endl;
```

```
return 0;  
}
```

Output:

A screenshot of a terminal window with a dark background. The title bar at the top shows the file path 'C:\AyushSR\crypto\lab11.exe' and standard window controls (close, maximize, minimize). The terminal text is as follows:

```
Enter a and n : 3 11  
  
---Extended Euclidean Algorithm---  
GCD = 1  
Multiplicative Inverse in Z11 = 4
```


Lab12: Source code

```
#include <bits/stdc++.h>

using namespace std;

long long powerMod(long long a,long long b,long long mod){

long long result=1;

a%=mod;

while(b>0){

if(b%2==1)

result=(result*a)%mod;

a=(a*a)%mod;

b/=2;

}

return result;

}

bool fermatTest(long long n){

if(n<4)

return n==2 || n==3;

int bases[]={2,3,5,7};

for(int i=0;i<4;i++){

if(bases[i]>=n)

continue;

if(powerMod(bases[i],n-1,n)!=1)

return false;

}

return true;

}

bool millerRabin(long long n){

if(n<2)
```

```

return false;

if(n==2 | | n==3)

return true;

if(n%2==0)

return false;

long long d=n-1;

int s=0;

while(d%2==0){

d/=2;

s++;

}

int bases[]={2,3,5,7,11};

for(int i=0;i<5;i++){

long long a=bases[i];

if(a>=n)continue;

long long x=powerMod(a,d,n);

if(x==1 | | x==n-1)

continue;

bool witness=true;

for(int r=1;r<s;r++){

x=(x*x)%n;

if(x==n-1){

witness=false;

break;

}

}

if(witness)

return false;

```

```

}

return true;

}

int main(){

long long n;

cout<<"Enter number : ";

cin>>n;

if(n<2)return cout<<"Number must be >= 2",0;

cout<<endl<<"---Fermat & Miller-Rabin Primality Tests---"<<endl;

cout<<"Fermat Test : ";

if(fermatTest(n))

cout<<"Probably Prime"<<endl;

else

cout<<"Composite"<<endl;

cout<<"Miller-Rabin Test : ";

if(millerRabin(n))

cout<<"Probably Prime";

else

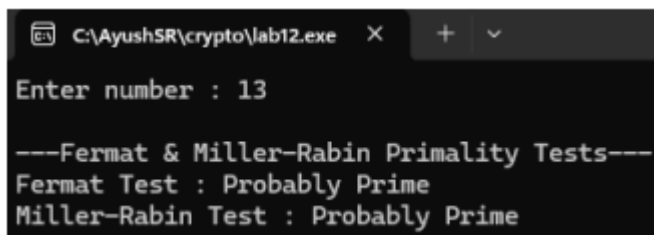
cout<<"Composite";

return 0;

}

```

Output:



```

C:\AyushSR\crypto\lab12.exe
Enter number : 13

---Fermat & Miller-Rabin Primality Tests---
Fermat Test : Probably Prime
Miller-Rabin Test : Probably Prime

```

Lab13: Source code

```
#include <bits/stdc++.h>

using namespace std;

long long gcdValue(long long a,long long b){
    while(b!=0){
        long long t=a%b;
        a=b;
        b=t;
    }
    return a;
}

long long powerMod(long long a,long long b,long long mod){
    long long result=1;
    a%=mod;
    while(b>0){
        if(b%2==1)result=(result*a)%mod;
        a=(a*a)%mod;
        b/=2;
    }
    return result;
}

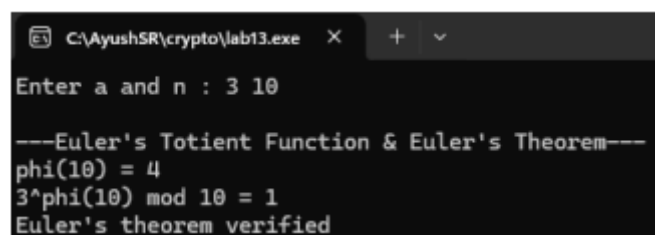
long long phi(long long n){
    long long result=n;
    for(long long p=2;p*p<=n;p++){
        if(n%p==0){
            while(n%p==0)
                n/=p;
            result-=result/p;
        }
    }
    return result;
}
```

```

}
}
if(n>1)
result-=result/n;
return result;
}
int main(){
long long a,n;
cout<<"Enter a and n : ";
cin>>a>>n;
long long phiValue=phi(n);
cout<<endl<<"---Euler's Totient Function & Euler's Theorem---"<<endl;
cout<<"phi("<<n<<" ) = "<<phiValue<<endl;
if(gcdValue(a,n)==1){
cout<<a<<"^phi("<<n<<" ) mod "<<n<<" = "<<powerMod(a,phiValue,n)<<endl;
if(powerMod(a,phiValue,n)==1)
cout<<"Euler's theorem verified"<<endl;
else
cout<<"Euler's theorem not verified"<<endl;
}
else cout<<"GCD("<<a<<","<<n<<") != 1, so Euler's theorem not applicable";
return 0;
}

```

Output:



```

C:\AyushSR\crypto\lab13.exe
Enter a and n : 3 10

---Euler's Totient Function & Euler's Theorem---
phi(10) = 4
3^phi(10) mod 10 = 1
Euler's theorem verified

```

Lab14: Source code

```
#include <bits/stdc++.h>

using namespace std;

long long powerMod(long long a,long long b,long long mod){

long long result=1;

a%=mod;

while(b>0){

if(b%2==1)

result=(result*a)%mod;

a=(a*a)%mod;

b/=2;

}

return result;

}

bool isPrime(long long n){

if(n<2)

return false;

for(long long i=2;i*i<=n;i++)

if(n%i==0)

return false;

return true;

}

vector<long long> primeFactors(long long n){

vector<long long> factors;

for(long long i=2;i*i<=n;i++){

if(n%i==0){

factors.push_back(i);

while(n%i==0)n/=i;

}
```

```

}
}
if(n>1)factors.push_back(n);
return factors;
}

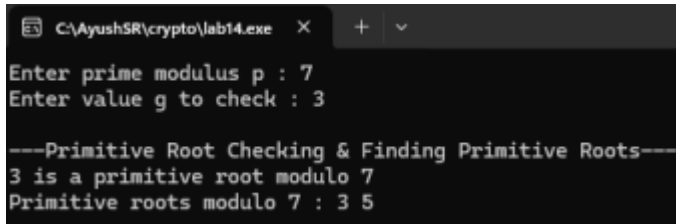
bool isPrimitiveRoot(long long g,long long p,vector<long long> factors){
    long long phi=p-1;
    for(int i=0;i<(int)factors.size();i++)
        if(powerMod(g,phi/factors[i],p)==1)
            return false;
    return true;
}

int main(){
    long long p,g;
    cout<<"Enter prime modulus p : ";
    cin>>p;
    cout<<"Enter value g to check : ";
    cin>>g;
    cout<<endl<<"---Primitive Root Checking & Finding Primitive Roots---"<<endl;
    if(!isPrime(p))
        return cout<<"p must be prime",0;
    vector<long long> factors=primeFactors(p-1);
    if(isPrimitiveRoot(g,p,factors))
        cout<<g<<" is a primitive root modulo "<<p<<endl;
    else
        cout<<g<<" is not a primitive root modulo "<<p<<endl;
    cout<<"Primitive roots modulo "<<p<<" : ";
    for(long long i=2;i<p;i++)

```

```
if(isPrimitiveRoot(i,p,factors))  
    cout<<i<<" ";  
return 0;  
}
```

Output:



```
C:\AyushSR\crypto\lab14.exe X + v  
Enter prime modulus p : 7  
Enter value g to check : 3  
  
---Primitive Root Checking & Finding Primitive Roots---  
3 is a primitive root modulo 7  
Primitive roots modulo 7 : 3 5
```


Lab15: Source code

```
#include <bits/stdc++.h>

using namespace std;

long long powerMod(long long a,long long b,long long mod){

long long result=1;

a%=mod;

while(b>0){

if(b%2==1)

result=(result*a)%mod;

a=(a*a)%mod;

b/=2;

}

return result;

}

int main(){

long long p,g,a,b;

cout<<"Enter prime p and primitive root g : ";

cin>>p>>g;

cout<<"Enter private key of Alice : ";

cin>>a;

cout<<"Enter private key of Bob : ";

cin>>b;

long long A=powerMod(g,a,p);

long long B=powerMod(g,b,p);

long long sharedAlice=powerMod(B,a,p);

long long sharedBob=powerMod(A,b,p);

cout<<endl<<"---Diffie-Hellman Key Exchange---"<<endl;

cout<<"Alice public value = "<<A<<endl;
```

```
cout<<"Bob public value = "<<B<<endl;

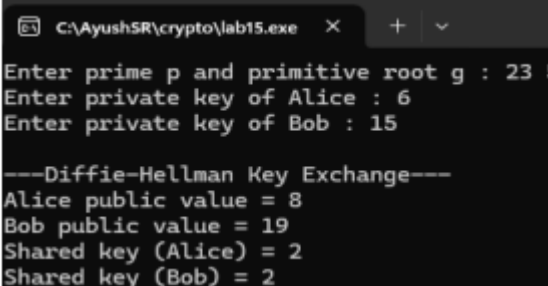
cout<<"Shared key (Alice) = "<<sharedAlice<<endl;

cout<<"Shared key (Bob) = "<<sharedBob;

return 0;

}
```

Output:



```
C:\AyushSR\crypto\lab15.exe X + v
Enter prime p and primitive root g : 23
Enter private key of Alice : 6
Enter private key of Bob : 15

---Diffie-Hellman Key Exchange---
Alice public value = 8
Bob public value = 19
Shared key (Alice) = 2
Shared key (Bob) = 2
```

Lab16: Source code

```
#include <bits/stdc++.h>

using namespace std;

long long gcdValue(long long a,long long b){
    while(b!=0){
        long long t=a%b;
        a=b;
        b=t;
    }
    return a;
}

long long powerMod(long long a,long long b,long long mod){
    long long result=1;
    a%=mod;
    while(b>0){
        if(b%2==1)
            result=(result*a)%mod;
        a=(a*a)%mod;
        b/=2;
    }
    return result;
}

long long extendedGCD(long long a,long long b,long long &x,long long &y){
    if(b==0){
        x=1;
        y=0;
        return a;
    }
```

```

long long x1,y1;

long long gcd=extendedGCD(b,a%b,x1,y1);

x=y1;

y=x1-(a/b)*y1;

return gcd;

}

long long modInverse(long long a,long long mod){

long long x,y;

long long gcd=extendedGCD(a,mod,x,y);

if(gcd!=1)

return -1;

return (x%mod+mod)%mod;

}

int main(){

long long p,q;

cout<<"Enter two primes p and q : ";

cin>>p>>q;

long long n=p*q;

long long phi=(p-1)*(q-1);

cout<<endl<<"---RSA Key Generation---"<<endl;

if(n<=25)

return cout<<"Choose larger primes",0;

long long e=2;

while(e<phi&&gcdValue(e,phi)!=1)e++;

long long d=modInverse(e,phi);

cout<<"Public key = ("<<e<<","<<n<<")"<<endl;

cout<<"Private key = ("<<d<<","<<n<<")"<<endl;

cin.ignore();

```

```

string message;

cout<<"Enter message : ";

getline(cin,message);

cout<<"Encrypted : ";

for(char ch:message){

ch=toupper(ch);

if(ch==' ')

cout<<" / ";

else if(ch>='A'&&ch<='Z'){

long long m=ch-'A';

cout<<powerMod(m,e,n)<<" ";

}}cout<<endl;

cout<<"Decrypted : ";

for(char ch:message){

ch=toupper(ch);

if(ch==' ')

cout<<" ";

else if(ch>='A'&&ch<='Z'){

long long m=ch-'A';

long long c=powerMod(m,e,n);

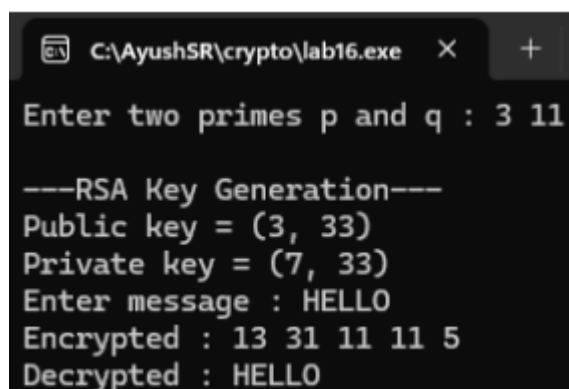
long long plain=powerMod(c,d,n);

cout<<char(plain+'A');

}}return 0;}

```

Output:



```

C:\AyushSR\crypto\lab16.exe
Enter two primes p and q : 3 11

---RSA Key Generation---
Public key = (3, 33)
Private key = (7, 33)
Enter message : HELLO
Encrypted : 13 31 11 11 5
Decrypted : HELLO

```

Lab17: Source code

```
#include <bits/stdc++.h>

using namespace std;

long long powerMod(long long a,long long b,long long mod){

long long result=1;

a%=mod;

while(b>0){

if(b%2==1)

result=(result*a)%mod;

a=(a*a)%mod;

b/=2;

}

return result;

}

long long extendedGCD(long long a,long long b,long long &x,long long &y){

if(b==0){

x=1;

y=0;

return a;

}

long long x1,y1;

long long gcd=extendedGCD(b,a%b,x1,y1);

x=y1;

y=x1-(a/b)*y1;

return gcd;

}

long long modInverse(long long a,long long mod){

long long x,y;
```

```

long long gcd=extendedGCD(a,mod,x,y);
if(gcd!=1)
return -1;
return (x%mod+mod)%mod;
}
int main(){
long long p,g,x,k,m;
cout<<"Enter prime p and primitive root g : ";
cin>>p>>g;
cout<<"Enter private key x : ";
cin>>x;
cout<<"Enter random key k : ";
cin>>k;
cout<<"Enter message m (m < p) : ";
cin>>m;
long long y=powerMod(g,x,p);
long long c1=powerMod(g,k,p);
long long c2=(m*powerMod(y,k,p))%p;
long long s=powerMod(c1,x,p);
long long sInverse=modInverse(s,p);
long long decrypted=(c2*sInverse)%p;
cout<<endl<<"---ElGamal Cryptographic System---"<<endl;
cout<<"Public key (p,g,y) = ("<<p<<","<<g<<","<<y<<")"<<endl;
cout<<"Ciphertext = ("<<c1<<","<<c2<<")"<<endl;
cout<<"Decrypted message = "<<decrypted;
return 0;
}

```

Output:

```
C:\AyushSR\crypto\lab17.exe X + v
Enter prime p and primitive root g : 23 5
Enter private key x : 6
Enter random key k : 3
Enter message m (m < p) : 10

---ElGamal Cryptographic System---
Public key (p,g,y) = (23,5,8)
Ciphertext = (10,14)
Decrypted message = 10
```


Lab18: Source code

```
#include <bits/stdc++.h>

using namespace std;

bool hasSuspiciousExtension(string name){

    string exts[]={" .exe", ".bat", ".vbs", ".js", ".scr", ".com"};

    for(int i=0;i<6;i++)

        if(name.size()-exts[i].size()==0)

            if(name.substr(name.size()-exts[i].size())==exts[i])

                return true;

    return false;

}

int main(){

    int n;

    cout<<"Enter number of sandbox file names : ";

    cin>>n;

    vector<string> files(n);

    cout<<"Enter file names : "<<endl;

    for(int i=0;i<n;i++)

        cin>>files[i];

    cout<<endl<<"Scan Report : "<<endl;

    for(int i=0;i<n;i++)

        if(hasSuspiciousExtension(files[i]))

            cout<<files[i]<<" -> suspicious"<<endl;

        else

            cout<<files[i]<<" -> normal"<<endl;

    string agentName;

    int copies;

    cout<<endl<<"Enter agent name : ";
```

```

cin>>agentName;

cout<<"Enter number of copies : ";

cin>>copies;

vector<string> sandboxCopies;

for(int i=1;i<=copies;i++)

sandboxCopies.push_back("copy_"+to_string(i)+"_"+agentName);

cout<<endl<<"Replication Log :"<<endl;

for(string s:sandboxCopies)

cout<<s<<endl;

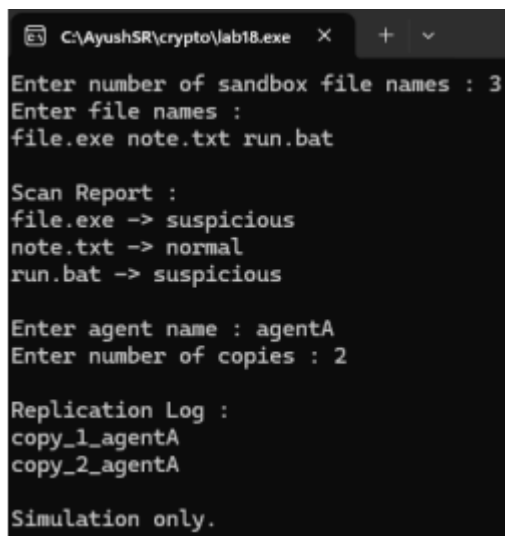
cout<<endl<<"Simulation only.";

return 0;

}

```

Output:



```

C:\AyushSR\crypto\lab18.exe
Enter number of sandbox file names : 3
Enter file names :
file.exe note.txt run.bat

Scan Report :
file.exe -> suspicious
note.txt -> normal
run.bat -> suspicious

Enter agent name : agentA
Enter number of copies : 2

Replication Log :
copy_1_agentA
copy_2_agentA

Simulation only.

```